

Reviewer Pack — Minimal Rank of Tropical Matrices and ACC^0 Lower Bounds for ...

Entry fa7c69f998d2 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 02:46:10 UTC

Minimal Rank of Tropical Matrices and ACC^0 Lower Bounds for Explicit Functions

Entry ID: fa7c69f998d2 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-22 02:46:10 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): ACC^0 Circuit Complexity

Statement:

{‘quantitative_relation’: ‘For every explicit function f in P with a polynomial-sized ACC^0 circuit, the minimal rank of its associated tropical matrix A is $\Theta(f(n))$ for some constant $c > 0$.’, ‘falsifier’: ‘Find an explicit function f in P with a polynomial-sized ACC^0 circuit such that the minimal rank of its associated tropical matrix A is $o(f(n))$.’, ‘constructive_mapping’: ‘Given an explicit function f , construct its tropical matrix A by encoding each input-output pair as a tropical vector and using tropical linear algebra operations.’}

Rationale (proposer’s reasoning):

{‘explanation’: ‘Tropical geometry provides a framework for studying the structure of polynomials over the max-plus semiring. By linking this to ACC^0 circuit complexity, we may uncover new insights into the limits of efficient computation.’, ‘bridge_exposure’: ‘This conjecture bridges tropical geometry with computational complexity by proposing a quantitative relationship between the rank of a tropical matrix and the size of an ACC^0 circuit.’}

Taxonomy category: SOS_HIERARCHY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c3cd9ac5aa1e9468

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for all explicit functions f in P with a polynomial-sized ACC^0 circuit, the ratio of the minimal rank of its tropical matrix A to the size of the ACC^0 circuit is greater than or equal to some constant $c > 0$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal rank tropical matrices AND ACC⁰ circuit complexity - Tropical geometry INTEGRATED with ACC⁰ lower bounds - Polynomial-sized ACC⁰ circuits AND tropical matrix minimal rank

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_polynomial(n):
    coefficients = [random.randint(0, 10) for _ in range(n + 1)]
    return lambda x: sum(c * (x ** i) for i, c in enumerate(coefficients))

def generate_acc0_circuit(f, n):
    inputs = [i for i in range(-n, n + 1)]
    outputs = f(inputs)
    acc0_circuit_size = len(outputs)
    return acc0_circuit_size

def tropical_matrix_rank(matrix):
    m, n = len(matrix), len(matrix[0])
    rank = 0
    for col in range(n):
        pivot_row = None
        for row in range(m):
            if matrix[row][col] != float('-inf'):
                pivot_row = row
                break
        if pivot_row is not None:
            rank += 1
            for i in range(m):
                if i != pivot_row and matrix[i][col] != float('-inf'):
                    factor = matrix[i][col] / matrix[pivot_row][col]
                    for j in range(n):
                        matrix[i][j] -= factor * matrix[pivot_row][j]
```

```

    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    total_metric_value = 0
    instances_tested = 0

    for n in n_values:
        f = generate_polynomial(n)
        acc0_circuit_size = generate_acc0_circuit(f, n)

        if acc0_circuit_size == 0:
            continue

        matrix = [[float('-inf')] * (n + 1) for _ in range(acc0_circuit_size)]
        for i in range(acc0_circuit_size):
            x = i - acc0_circuit_size // 2
            y = f(x)
            matrix[i][x] = y

        rank = tropical_matrix_rank(matrix)

        if rank == 0:
            continue

        instances_tested += 1
        total_metric_value += rank / acc0_circuit_size

    if instances_tested == 0:
        return {
            "metric_name": "rank_ratio",
            "metric_value": None,
            "instances_tested": 0,
            "conjecture_holds": False,
            "counterexample": "no_valid_instances"
        }

    mean_metric_value = total_metric_value / instances_tested
    conjecture_holds = mean_metric_value >= 1

    return {
        "metric_name": "rank_ratio",
        "metric_value": mean_metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else "not_enough_evidence"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:

```

```

result = run_trial(seed)
print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_
results.append(result)

if all(r[\"conjecture_holds\"] for r in results):
    print(f"RESULT: SUPPORTED mean={sum(r['metric_value'] for r in results) / len(results)} st
elif any(not r[\"conjecture_holds\"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result[\"conj
    print(f"RESULT: FALSIFIED counterexample=\\\"not_enough_evidence\\\" first_failing_seed={first
else:
    print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0a28b101.py", line 100, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0a28b101.py", line 55, in run_trial
    acc0_circuit_size = generate_acc0_circuit(f, n)
                        ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0a28b101.py", line 24, in generate_acc0
    outputs = f(inputs)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0a28b101.py", line 20, in <lambda>
    return lambda x: sum(c * (x ** i) for i, c in enumerate(coefficients))
                        ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0a28b101.py", line 20, in <genexpr>
    return lambda x: sum(c * (x ** i) for i, c in enumerate(coefficients))
                        ~~~~~
TypeError: unsupported operand type(s) for ** or pow(): 'list' and 'int'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from verifying the conjecture's support condition. | next: Investigate and fix the error in the test code to allow for a proper verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13599
2	propose	ollama_remote	glm4:latest	0	0	13267
3	preregistration	ollama_remote	glm4:latest	0	0	9678
4	novelty	ollama_remote	glm4:latest	0	0	8304
5	novelty	ollama_remote	glm4:latest	0	0	9441
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16175
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7829
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13612
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11198
10	judge	ollama_remote	glm4:latest	0	0	13327

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 116430 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/fa7c69f998d2.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/fa7c69f998d2.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/fa7c69f998d2.tar.gz` (if generated)